

Action Prefix System

(\$. .)

A prompt whose **first non-blank line** starts with a registered action token is not an ordinary prompt: the token is **replaced** by that action's registered `expansion` text, the action's `rules` apply, and the remainder of the prompt is the actual task, executed under the expanded instruction.

The registry — `constitution/actions/registry.yaml` (override with `$HELIX_ACTION_REGISTRY`) — is the **single source of truth**. It is DATA: adding an action is a registry row, never a code change.

The six equivalent forms

All six resolve to the SAME action, the SAME expansion, the SAME execution:

#	Form	Example
1	<code>ACTION :: <rest></code>	<code>BACKGROUND :: refactor the parser</code>
2	<code>PREFIX::ACTION :: <rest></code>	<code>DEFAULT::BACKGROUND :: refactor the parser</code>
3	<code>/ACTION <rest></code>	<code>/background refactor the parser</code>
4	<code>/PREFIX::ACTION <rest></code>	<code>/DEFAULT::BACKGROUND refactor the parser</code>
5	<code>ACTION ---> <rest></code>	<code>BACKGROUND ---> refactor the parser</code>
6	<code>ACTION: <rest></code> — single colon	<code>BUG: subtitles render one frame late</code>

`PREFIX` is a **namespace**; the reserved default namespace is `DEFAULT`. An action runs with or without it.

Form 6 is the one most people type naturally (`BUG: ...` , `TASK: ...`) — a colon and exactly one space, no space *before* the colon.

Form `is REGISTERED-ACTION-ONLY` by construction

Unlike the other five, an unknown single-colon token is a **silent no-op** — it never asks for clarification and never resolves to a sub-system (`single_colon_registered_only: true`). This is deliberate: ordinary prose like `TODD: fix this later` , `NOTE: see below` , or `WARNING: slow path` **MUST NOT** trigger a directive or a clarifying question. Only a token that is *already registered* activates in form 6.

Grammar (all of these hold)

- **Anchored to the first non-blank line only.** A token mid-prose NEVER matches.
- **UPPERCASE only** — `[A-Z][A-Z0-9_]*` . Lowercase never matches. (File and directory names stay lowercase snake_case per §11.4.29 — the *token* is upper.)
- **Namespace separator** `::` carries **NO** surrounding spaces (`PREFIX::ACTION`), which is what distinguishes it from the **body separator** `" :: "` (exactly one ASCII space each side). That spacing rule is why C++ `Foo::Bar` , YAML `key: value` , and URLs never false-trigger.
- **Body separators:** `" :: "` (forms 1/2), `" ---> "` (form 5), one space (forms 3/4), and `" : "` — colon + exactly one space, **no space before the colon** — (form 6).
- **Stacked prefixes** apply outer-to-inner, left-to-right: `A :: B :: rest` expands `A` , re-scans, expands `B` , and the residual is the task.
- **Escape with a leading** `\` — `\BACKGROUND :: x` , `\BACKGROUND ---> x` , `\BACKGROUND x` are literal: strip the backslash, NO expansion. Use this to *discuss* an action without invoking it.

The conflict rule – never silently shadow a host command

A bare `/ACTION` (form 3) is honored **only** when it does not collide with a built-in/host slash command. Collisions are declared as data in the registry (`slash_bare: auto` + `slash_conflicts: [...]`).

- `/PREFIX::ACTION` (form 4) is **ALWAYS unambiguous** and always honored.
- On **Claude Code**, plugin namespacing is the form-4 escape: `/helix:<name>` always resolves to the action. A third, collision-free form works on every agent: `/default-<name>` .
- Worked example: `BUG` declares `slash_conflicts: [bug]` because Claude Code ships a built-in `/bug` . So the bare `/bug` goes to the **host built-in**, and the action is invoked as `/helix:bug` or `/default-bug` . The plugin does not fight the host and does not hide it.

The iron rule for unknown tokens (`$ . .`)

A token that matches the grammar shape but is **NOT registered** is **NEVER** silently expanded. In forms 1–5, do exactly one of:

1. Ask which registered action was meant (§11.4.66 / §11.4.105), or
2. Treat the line literally as an ordinary prompt.

Exception — form 6 only: an unregistered single-colon token is a **silent no-op** by design (see above). `TODO: ...` must never provoke a question.

Never invent an expansion. Inventing one is a §11.4.6 no-guessing violation. A prompt that does not satisfy the grammar is an ordinary prompt — the system is a silent no-op, which is the correct behaviour.

Sub-system shortcuts

A grammar-shaped UPPERCASE token that is not a *behavioral* action MAY name an incorporated sub-system/submodule (from the registry `subsystems:` catalogue, or auto-derived by recursive `.gitmodules` discovery from the invoking

project's root). It expands to a **sub-system context injection** (repository + org + checkout location + the §11.4.28 decoupling/equal-codebase rules + §11.4.37 fetch-first + §11.4.113 no-force-push), and the remainder is the task on that sub-system. A behavioral **action always wins** a token collision. An **ambiguous** token (resolving to >1 sub-system) is DROPPED → falls through to ASK.

Adding a new action

1. Add one row under `actions:` in `constitution/actions/registry.yaml` (`name`, `summary`, `expansion`, `rules`, `slash_bare`, `slash_conflicts`).
2. Re-run `bash constitution/scripts/generate_agent_prefix_commands.sh`.
3. Every agent's slash command appears: Claude Code plugin command, Gemini/Qwen `.toml`, Codex prompt `.md`. **No code changes.**

The generator + installer are re-run automatically on every constitution pull (§11.4.164 `post_update_hook.sh`), so a new directive is live out of the box.

Reference

- Canonical: `constitution/Constitution.md` §11.4.140.
- Recognition contract: `constitution/actions/recognition_instruction.md`.
- Resolver library: `constitution/scripts/action_prefix_lib.sh`.
- Manuals: `constitution/docs/actions/`.